

Forced reflow

Одной из незаметных, но значительных проблем с производительностью является принудительный синхронный перерасчёт макета (forced reflow/forced synchronous layout). Если проще, то это чтение данных о содержимом страницы сразу после изменения этого содержимого (стилей/вёрстки). Незаметной эта проблема является из-за того, что тормоза возникают когда пользователь уже видит картинку, т.е. проблему можно увидеть только если попытаться взаимодействовать со страницей в этот момент (или профилировать).

Не любое чтение приводит к перерасчёту макета, в основном это вещи, связанные с размерами элементов. Существует [список "опасных" свойств](#), созданный одним из инженеров Google.

При использовании нашего фреймворка такие проблемы встречаются достаточно редко: если работать строго через опции и состояние, то все изменения будут группироваться и применяться синхронно. Размеры элементов можно измерять в хуках жизненного цикла, которые вызываются в те моменты, когда нет каких-то неотрисованных изменений.

Проблемы возникают, когда замеры производятся в какие-то другие моменты: например, в коллбеке таймера/интервала/promise и т.п. Вызовы этих коллбеков могут попасть в тот момент, когда страница уже изменена, но ещё не была отрисована.

Для примера возьмём такую ситуацию: допустим, вы открываете карточку, и в это же время у вас идёт запрос за данными списка. Данные для списка приходят во время построения карточки, и после их прихода вы хотите получить текущие высоты элементов (например, для виртуального скроллинга, чтобы понять сколько элементов удалять).

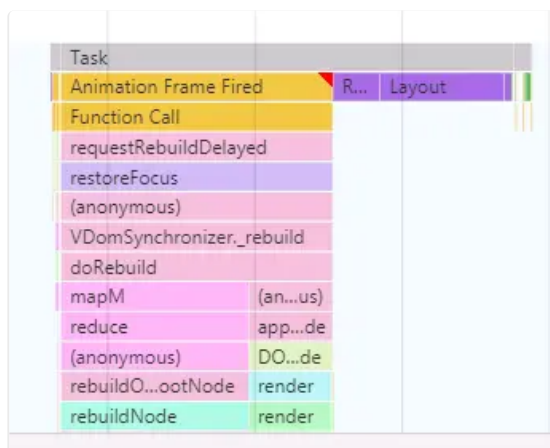
Демо-пример изображает подобную ситуацию: на нём есть две кнопки, одна из них скрывает/показывает список, другая включает/выключает замеры высоты. Для упрощения примера вместо карточки используется список, на контейнере которого переключается display: none, а замеры высоты производятся в таком участке кода:

▼ Пример кода

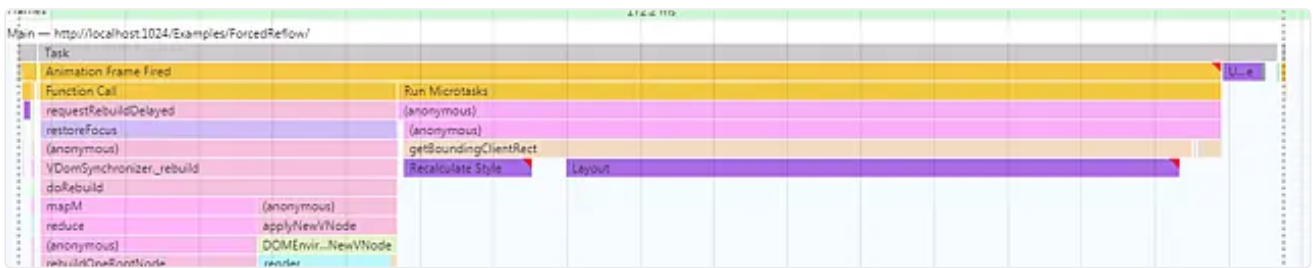
```
1 protected _beforeUpdate(): void {
2   if (this._enableMeasurements) {
3     Promise.resolve().then(() => {
4       this._itemHeights = Array.from(
5         document.querySelector('.DebuggingExamples-
6         ForcedReflow__list').children
7         ).map((child) => child.getBoundingClientRect().height);
8     });
9   }
```

[См. демо-пример](#)

Если мы запустим нативный профилировщик Chrome и нажмём на кнопку "Показать список", то получим такую картину:



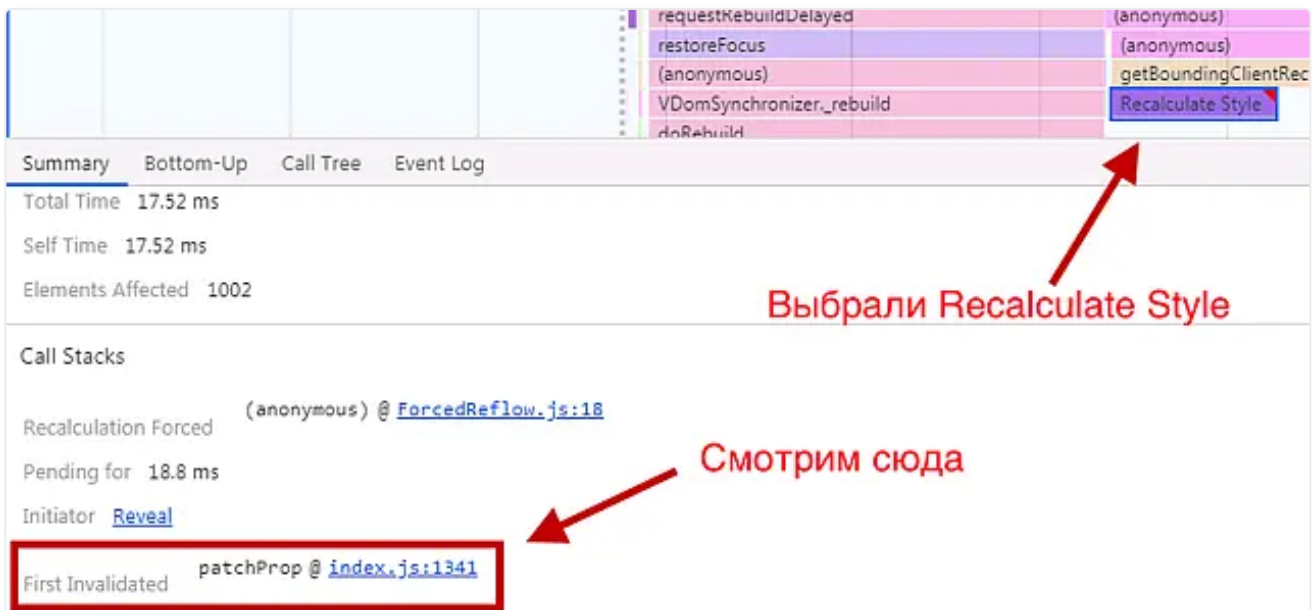
Обращать внимание здесь нужно на фиолетовые участки, а именно на Recalculate Style и Layout. Здесь они у нас находятся наверху стека, а значит проблем нет. Теперь скроем список, нажмём на кнопку "Запустить замеры" и опять нажмём на кнопку "Показать список" с включённым профилировщиком.



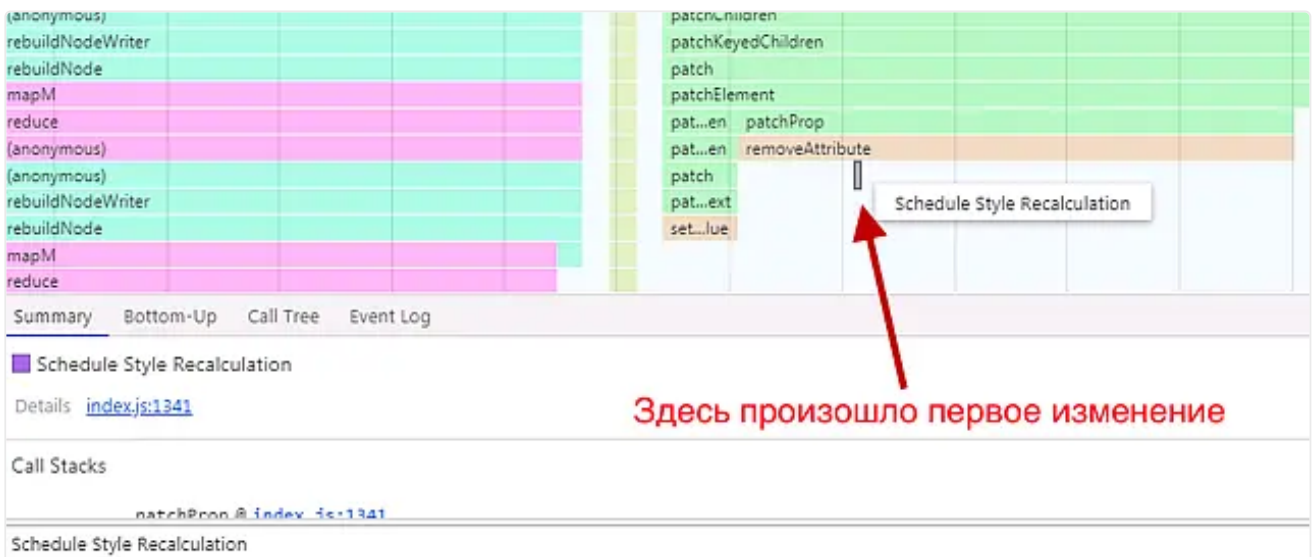
Здесь видно, что Relacalculate Style и Layout теперь не наверху стека и занимают много времени. Также на них есть красный флажок, и если навести курсор или выбрать любой из этих элементов, то появится следующее предупреждение: "Forced reflow is a likely performance bottleneck."

Теперь мы научились определять где происходит forced reflow, но само по себе это знание ничего не даст – код внутри коллбека ничего не меняет, а только читает, так что проблема не в нём. Чтобы найти место, где происходили изменения, существует два способа:

Первый способ – выбрать проблемный Recalculate Style или Layout и найти строчку First invalidated в описании – это и будет местом изменения.



Второй способ это поиск внутри профиля по фразе "Schedule Style Recalculation". Это покажет конкретное место в профиле, где произошли изменения, и даст возможность посмотреть на стек.



Большая часть таких проблем решается выносом измерения размеров в хуки жизненного цикла, но не всегда это легко сделать, и не всегда это вообще можно сделать. Сначала разберём ситуацию, когда замер происходит в коллбеке, который вызывается асинхронно. Здесь две ситуации:

1. Синхронизация идёт.
2. Синхронизация не идёт.

В первом случае нужно обязательно дожидаться окончания синхронизации, не только из-за проблем с производительностью, а ещё и из-за того, что размеры элементов могут измениться после неё. Как определить, что сейчас идёт синхронизация? Пока самый надёжный способ это флаг на инстансе – в [_beforeUpdate\(\)](#) устанавливать его в одно состояние, а в [_afterUpdate](#) – в другое.

Второй случай на первый взгляд кажется достаточно простым – если синхронизация не идёт, то можно делать всё что угодно. К сожалению, не всё так просто – синхронизация может идти, просто ваш контрол не является её частью. Поэтому самым надёжным вариантом является ручной запуск синхронизации и выполнение замеров после неё. Если у вас при синхронизациях не изменяется состояние, то это не добавит сильных накладных расходов. А если меняется, то у вас существуют другие проблемы, кроме forced reflow.

Теперь разберём другую частую ситуацию: вы делаете замеры в правильном месте, страницу не меняете, а forced reflow все равно есть – значит это делает кто-то перед вами. Правильным решением тут будет найти виновника, и устранять проблему уже там.

Наконец, разберём ситуации, когда forced reflow вообще не избежать. Для этого возьмём два примера:

1. Восстановление скrolла в списке.
2. Использование сторонней библиотеки, которая напрямую работает с DOM (например, tinymce).

В списках бывает ситуация, когда теряется позиция скrolла, например, если в списке включен виртуальный скrolлинг. Чтобы восстановить позицию, нужно понять насколько проскроллился список и скорректировать scrollTop. Если это делать в [_afterUpdate](#), то пользователь увидит моргание – сначала список отрисовывается с неправильным положением скrolла, а потом скакнёт в нужное место. Чтобы избежать этой проблемы, восстанавливать скrolл приходится до того, как браузер успел отрисовать картинку. Для этого у нас есть хук [_afterRender\(\)](#). Он выполняется синхронно после изменения DOM, так что любое чтение здесь гарантированно приведёт к forced reflow. Зато пользователь не увидит моргания, так что в этой ситуации это нормальное решение.

Теперь рассмотрим второй пример – сторонние библиотеки. Обычно работа с ними происходит где-нибудь в [_afterMount\(\)](#). Казалось бы, браузер уже всё отрисовал, всё нормально. Но если после выполнения нашего кода выполнится чей-то ещё [_afterMount\(\)](#), и там будет чтение, то это приведёт к forced reflow. Для избежания таких ситуаций лучше всю работу со сторонними библиотеками выносить в отдельные задачи, например, через requestAnimationFrame.